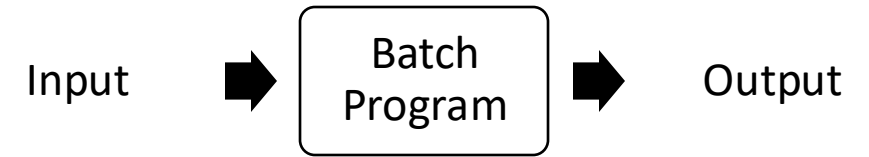


IO Actions

CSC345: Programming Languages and Paradigms

Early Days of Computing



- Historically, in early days of computing, most programs were batch programs
 - Run in isolation from user
 - Maximizing the time computer was performing useful work
- In early days of computing programs were modeled as pure functions:

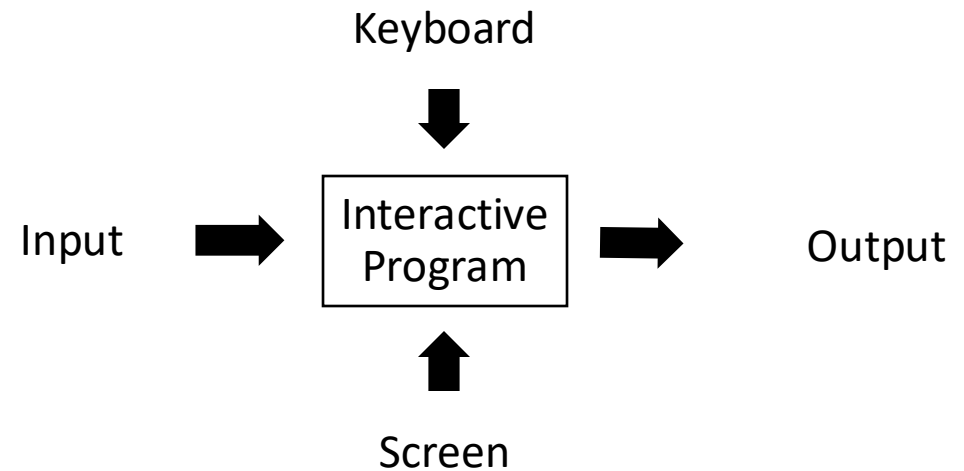
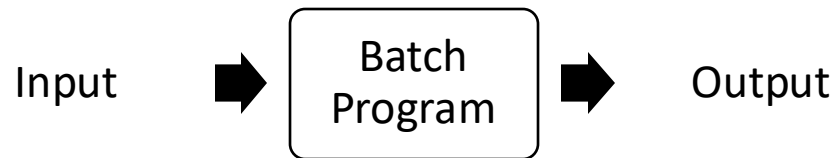
Inputs (w/ explicit arguments)

produce

Outputs (w/ explicit results)

Interactive Programs

- Modern era of computing: most programs are interactive programs



Interactive Programs and Side Effects

- ***Prompt:*** why are interactive programs a problem for functional P/Ls?
- **Interactive programs, by their very nature, require side effects.**
- Suppose we have a function that reads an integer from the user.
 - Value of `inputInt` is the value read in.

`inputInt :: Integer`

- Then, it is possible to write functions which will have an unexpected outcome

`inputDiff :: Integer`

`inputDiff = inputInt - inputInt`

Interactive Programs and Side Effects

`inputInt :: Integer`

`inputDiff :: Integer`

`inputDiff = inputInt – inputInt`

- Function can have an unexpected outcome
- Expression now is no longer determined by looking at the meaning of its parts
- Cannot give a meaning to `inputInt` without knowing where it occurs in a program
- IO is a “thorny” issue for functional programmers
 - Various approaches have been tried throughout the years
- IO in Haskell: view of actions happening in a sequence

Abstract Idea: I/O in Haskell

- *Abstract Representation of an Interactive Haskell Program:*

`type IO = World -> World`

- `World`: abstract type whose values represent states of the world
- Allows for side effects

- *Abstract Action of Performing a Side Effect and Returning a Value*

`type IO a = World -> (a, World)`

- `(a, World)`: “action” (w/ side effect) has occurred and the world is in a new state
- `a`: the value we wish to return

Actual: I/O in Haskell

~~type IO = World -> World~~

- **World**: abstract for explanatory purposes
- The **World** type doesn't actually exist (don't try to type it in)
- **IO a** is indeed defined in Haskell!
 - **IO Char**: the type of actions that return a character
 - **IO ()**: actions performed but return value isn't important

Basic Actions

`getChar :: IO Char`

- Reads a single character and returns the character as its result value

`putChar :: Char -> IO ()`

- Writes the character argument to the screen and returns no result value

`return :: a -> IO a`

- An IO action that does no IO but does return a value
- Provides a bridge from pure expressions with no side effects to impure actions with side effects
 - One-way bridge
 - The result of an impure action cannot be passed to a pure function unless it is called from within a do sequence (to be defined in a few slides)

More Basic Actions

`getLine :: IO String`

`putStr :: String -> IO ()`

`putStrLn :: String -> IO ()`

do Sequences

- A sequence of IO actions that can be combined into a single composite action

General Form:

```
do v1 <- a1  
  v2 <- a2  
  ...  
  vn <- an  
  return (f v1 v2 ... vn)
```

- Apply actions in-turn, naming result values, and at the end optionally applying some function to combine the results into a single value (“to be returned”)
- Saving values produced by actions is not required
- Optional return

Example

```
readThreeChars :: IO (Char, Char)
```

```
readThreeChars = do x <- getChar  
                  getChar  
                  y <- getChar  
                  return (x,y)
```

- Function that reads three characters, discards the second, and returns the first and third.

Big Idea

do sequences provide a small, imperative P/L for writing I/O programs on top of Haskell, without compromising the functional model of Haskell itself.

Exploring How Basic Actions are Defined in the Haskell Prelude (via function reuse)

How is `putStr :: String -> IO()` defined?

```
putStr :: String -> IO()
```

```
putStr [] = return ()
```

```
putStr (x:xs) = do putChar x  
                  putStr xs
```

How is `putStrLn :: String -> IO()` defined?

```
putStrLn :: String -> IO()
```

```
putStrLn str = do putStrLn str  
                 putStrLn "\n"
```

How is `getLine :: IO String` defined?

`getLine :: IO String`

```
getLine = do x <- getChar
           if x == '\n' then
             return []
           else
             do xs <- getLine
              return (x:xs)
```

`getLine :: IO String`

`putStr :: String -> IO ()`

`putStrLn :: String -> IO ()`

- These three functions are each already defined in the Haskell Standard prelude, but it was worthwhile to look at how they are coded.

- A potentially useful function that is not already defined:

`getInt :: IO Int`

A Quick Tangent

- We'll introduce class contexts soon in a future class.
- We've already briefly talked about the of the Show class and show function.
- The type signatures for the class `Show` with the function `show`, and the class `Read` with the function `read`:

`show :: Show a => a -> String`
`read :: Read a => String -> a`
- Note that both `show` and `read` are polymorphic and their type needs to be resolved.

Try it Out

show 99

read "123"::Int

(read "123")::Int

read ("123"::Int)

Back to I/O

How would `getInt :: IO Int` be defined? (It's not already defined for us.)

```
getInt :: IO Int
```

```
getInt = do line <- getLine  
         return (read line :: Int)
```

More Examples

An example w/ prompting and output.

```
strlen :: IO ()
```

```
strlen = do putStr "Enter a string: "  
           xs <- getLine  
           putStr "The string has "  
           putStr (show (length xs))  
           putStrLn " characters."
```

Observe the big idea: that we are calling the **pure function** `length` from within a do sequence using an argument that was saved as a result of an **IO action**.

The Following Example Uses **let** Statements

where construct

- **where** can only be used at the level of a function definition
 - We previously covered **where** in this course.
 - *One usage*: a function with guards
 - The scope of **where** extends over all the guards.

let statement

- **let** used wherever there is an expression
 - *One usage*:
`let {assignments} in {expression}`
 - The scope of **let** is only the current function, guard, expression, **subsequent statements in a do sequence**.

let Statements

- For our purposes, used inside of a **do** sequence.
- No `in` keyword (as in previous slide).

- *General Form:*

```
do statements  
...  
let var = expr  
...  
statements
```

- Assignments only in scope for any statements that follow in the **do** sequence.

Another Example 1

Double reversal of two lines.

```
reverse2lines :: IO ()
```

```
reverse2lines = do line1 <- getLine  
                  line2 <- getLine  
                  let rev1 = reverse line1  
                  let rev2 = reverse line2  
                  putStrLn rev2  
                  putStrLn rev1
```

Another Example 2

```
copy :: IO ()  
  
copy = do line <- getLine  
        putStrLn line  
        copy
```

This program would loop forever.

- *(Needs to be interrupted within GHCi by typing Ctrl-C.)*

This program is tail recursive.

- Function calls itself at end (“the tail”) and no computation is done after the return of the recursive call.
- Tail recursion is a special case of recursion.
- Efficient to implement and resemble a loop in an imperative P/L.
- (CSC416/565 topic “Compilers”: tail calls can be implemented without adding a new stack frame to the call stack.)

Prompt: what would `copy` look like in an imperative P/L?

Another Example 3

*“A counting variable loop”: a loop that copies **n** times, instead of forever (as in the previous example).*

Function Signature:

`copyN :: Integer -> IO ()`

Function Definition: